

**RANCANG BANGUN SISTEM DETEKSI KEBAKARAN RUMAH
BERBASIS IOT DAN ANDROID DENGAN MENGGUNAKAN
COMBO ESP32, ESP32-CAM, SERTA SENSOR MQ-135 DAN DHT11**

MANUAL BOOK

OLEH

- | | | |
|----|-----------------------------------|----------------------|
| 1. | Muhammad Rifandi Wira Kusuma S,Si | (200322615293) |
| 2. | Nugroho Adi Pramono, S.Si, M.Sc | (198303242006041003) |
| 3. | Ariko Yahya Setyawan, S.Si | (200322615247) |
| 4. | Dheanra Patiku, S.Pd | (210321606803) |
| 5. | Anugerah Agung Dwi Putra, S.Si | (200322615241) |
| 6. | Nunung Anggraini | (230322600948) |
| 7. | Nagata Dheta Permata Caesar | (240322603241) |
| 8. | Nayaka Dheta Permata Caesar | (250322621597) |



**UNIVERSITAS NEGERI MALANG
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
DEPARTEMEN FISIKA
JANUARI 2025**

LANGKAH PENGGUNAAN

A. Software dan Hardware

- Perangkat: 1× ESP32, 1× ESP32-CAM (OV2640), catu daya 5V, kabel USB/FTDI, Wi-Fi 2.4 GHz.
- Aplikasi: Arduino IDE.
- Akun: Bot Telegram aktif (**BOT_TOKEN** & **CHAT_ID**).

B. Source Coding

- **ESP32** : Sebuah code khusus untuk diupload di mikrokontroller ESP32 (baca DHT & MQ135, kirim teks via Serial1 ke ESP32-CAM)
- **ESP32-CAM** : Beberapa code khusus untuk diupload di mikrokontroller ESP32-CAM termasuk bagian coding camera_code.h dan camera_pins.h (konfigurasi kamera & Telegram)

C. Instalasi Singkat

1. Buka masing-masing kode di Arduino IDE.
2. Isi **SSID**, **Password Wi-Fi**, **BOT_TOKEN**, **CHAT_ID** (di file ESP32-CAM).
3. **Upload code ke ESP32** seperti biasa.
4. **Upload code ke ESP32-CAM**: masuk **mode flash** (IO0→GND saat reset), unggah, lalu lepaskan IO0 dan reset.

D. Cara Menyalakan & Uji

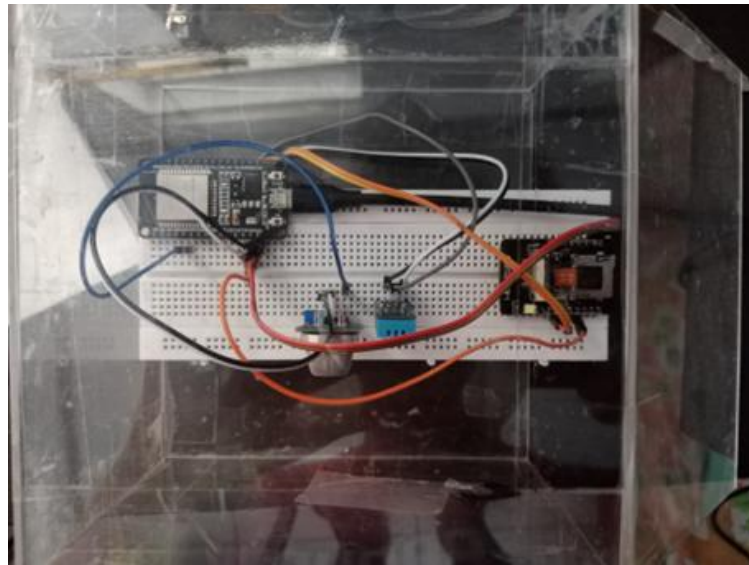
1. Nyalakan **ESP32** dan **ESP32-CAM**.
2. Buka Telegram, mulai chat dengan bot Anda → kirim **/start**.
3. Uji perintah:
 - **/conditions** → kirim nilai sensor terkini.
 - **/photo** → kirim foto dari ESP32-CAM.
 - **/flash** → nyalakan/matikan lampu kilat kamera.

E. Pakai Sehari-hari

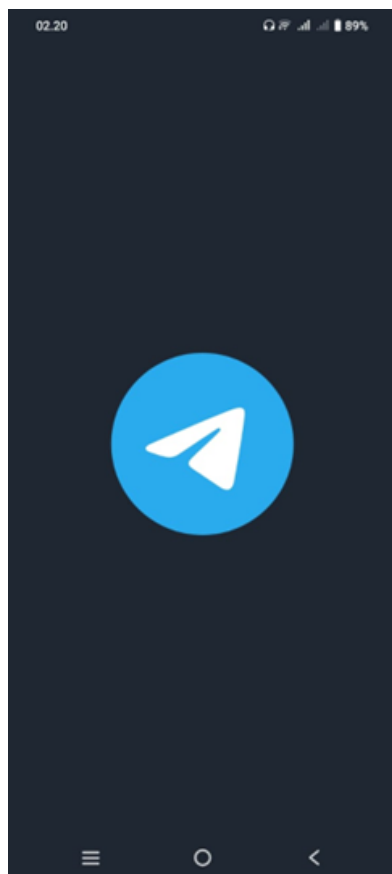
- Sistem memantau otomatis. Jika melewati ambang bahaya, bot mengirim **peringatan**.

Untuk cek kondisi dan bukti visual, gunakan **/conditions** dan **/photo**.

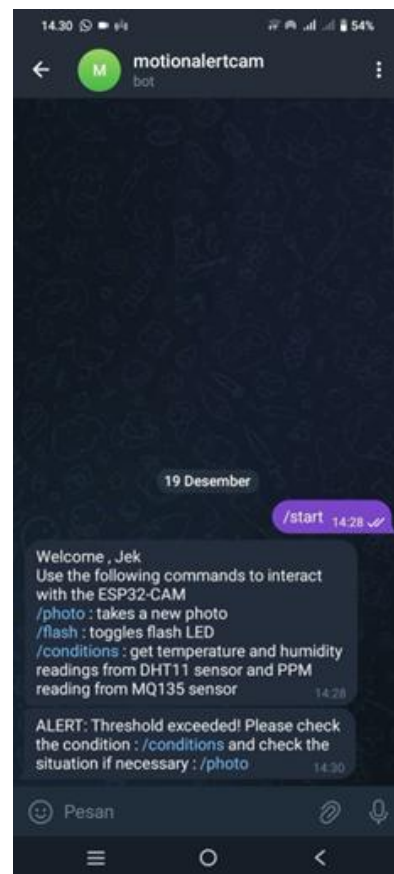
TAMPILAN SCREENSHOTS



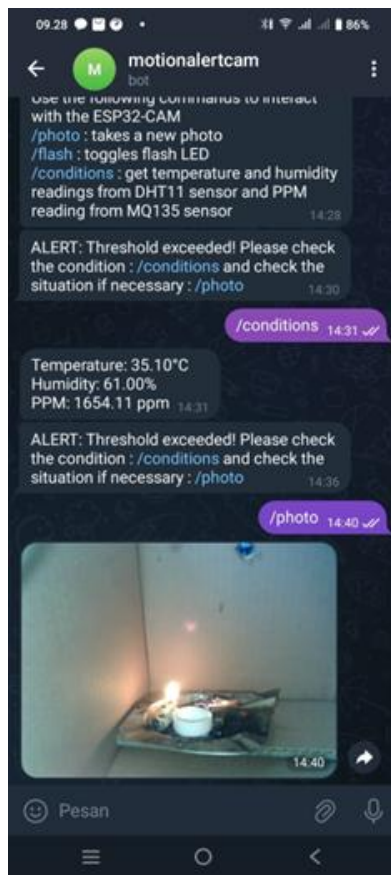
Gambar 1. Tampilan Prototype Sistem Deteksi Kebakaran



Gambar 2. Tampilan Aplikasi Android Telegram



Gambar 3. Tampilan Awal Pada Bot Telegram



Gambar 5. Tampilan Pengguna Cek Situasi dengan meminta foto menggunakan perintah “/photo”



Gambar 4. Tampilan Pengguna Cek Kondisi dengan meminta informasi menggunakan perintah “/conditions”



Gambar 6. Tampilan Notifikasi Aplikasi Telegram

POTONGAN SOURCE CODE

ESP32

```
#include <MQ135.h>
#include <DHT.h>
#include <HardwareSerial.h>
/* MQ135 + DHT Temp Sensor

Combination of the MQ135 air quality sensor and a DHT11/22
temperature sensor to accurately measure ppm values through the
library correction.
Uses the Adafruit DHT Sensor Library:
https://github.com/adafruit/DHT-sensor-library

Written by: https://github.com/NuclearPhoenixx/MQ135
*/
HardwareSerial MySerial(1);

#define PIN_MQ135 35 // MQ135 Analog Input Pin
#define DHTPIN 5 // DHT Digital Input Pin
#define DHTTYPE DHT11 // DHT11 or DHT22, depends on your sensor

MQ135 mq135_sensor(PIN_MQ135);
DHT dht(DHTPIN, DHTTYPE);

float temperature, humidity; // Temp and Humid floats, will be
measured by the DHT

// Define threshold limits
const float TEMP_THRESHOLD = 48.0; // Example threshold for
temperature
const float HUMID_THRESHOLD = 35.0; // Example threshold for
humidity
const float PPM_THRESHOLD = 200.0; // Example threshold for PPM

void setup() {
  Serial.begin(115200);
  MySerial.begin(9600, SERIAL_8N1, 16, 17);

  dht.begin();
}

void loop() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  // Check if any reads failed and exit early (to try again).
  if (isnan(humidity) || isnan(temperature)) {
    Serial.println(F("Failed to read from DHT sensor!"));
    return;
  }

  float rzero = mq135_sensor.getRZero();
  float correctedRZero = mq135_sensor.getCorrectedRZero(temperature,
```

```
humidity);
    float resistance = mq135_sensor.getResistance();
    float ppm = mq135_sensor.getPPM();
    float correctedPPM = mq135_sensor.getCorrectedPPM(temperature,
humidity);

    String sensorData = "Temperature: " + String(temperature) + "°C
Humidity: " + String(humidity) + "% PPM : " + String(correctedPPM)
+ " ppm";

    MySerial.println(sensorData);

    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.print("\t Humidity: ");
    Serial.print(humidity);

    Serial.print("\t MQ135 RZero: ");
    Serial.print(rzero);
    Serial.print("\t Corrected RZero: ");
    Serial.print(correctedRZero);
    Serial.print("\t Resistance: ");
    Serial.print(resistance);
    Serial.print("\t PPM: ");
    Serial.print(ppm);
    Serial.print("ppm");
    Serial.print("\t Corrected PPM: ");
    Serial.print(correctedPPM);
    Serial.println("ppm");

    delay(500);
}
```

ESP32-CAM

```
#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include "soc/soc.h"
#include "soc/rtc_cntl_reg.h"
#include "esp_camera.h"
#include <UniversalTelegramBot.h>
#include <ArduinoJson.h>
#include <HardwareSerial.h>

HardwareSerial MySerial(1);

const char* ssid = "VIVO";
const char* password = "22228888";

// Initialize Telegram BOT
String BOTtoken = "7453174459:AAHuCHsCnGkSvdsOQUyRCPs2Df83-03xtbg"; // your Bot Token (Get from Botfather)
String CHAT_ID = "1257831237";

bool sendPhoto = false;

WiFiClientSecure clientTCP;
UniversalTelegramBot bot(BOTtoken, clientTCP);

#define FLASH_LED_PIN 4
bool flashState = LOW;

// Variabel untuk membaca data sensor
String serialData = "";
float temperature = 0.0, humidity = 0.0, correctedPPM = 0.0;

// Checks for new messages every 1 second.
int botRequestDelay = 1000;
unsigned long lastTimeBotRan;

// Camera model AI Thinker
#define PWDN_GPIO_NUM    32
#define RESET_GPIO_NUM  -1
#define XCLK_GPIO_NUM    0
#define SIOD_GPIO_NUM    26
#define SIOC_GPIO_NUM    27

#define Y9_GPIO_NUM      35
#define Y8_GPIO_NUM      34
#define Y7_GPIO_NUM      39
#define Y6_GPIO_NUM      36
#define Y5_GPIO_NUM      21
#define Y4_GPIO_NUM      19
#define Y3_GPIO_NUM      18
#define Y2_GPIO_NUM       5
#define VSYNC_GPIO_NUM   25
#define HREF_GPIO_NUM    23
```

```

#define PCLK_GPIO_NUM      22

void configInitCamera() {
    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;
    config.pin_xclk = XCLK_GPIO_NUM;
    config.pin_pclk = PCLK_GPIO_NUM;
    config.pin_vsync = VSYNC_GPIO_NUM;
    config.pin_href = HREF_GPIO_NUM;
    config.pin_sscb_sda = SIOD_GPIO_NUM;
    config.pin_sscb_scl = SIOC_GPIO_NUM;
    config.pin_pwdn = PWDN_GPIO_NUM;
    config.pin_reset = RESET_GPIO_NUM;
    config.xclk_freq_hz = 20000000;
    config.pixel_format = PIXFORMAT_JPEG;
    config.grab_mode = CAMERA_GRAB_LATEST;

    // init with high specs to pre-allocate larger buffers
    if (psramFound()) {
        config.frame_size = FRAMESIZE_UXGA;
        config.jpeg_quality = 10; // 0-63 lower number means higher
quality
        config.fb_count = 1;
    } else {
        config.frame_size = FRAMESIZE_SVGA;
        config.jpeg_quality = 12; // 0-63 lower number means higher
quality
        config.fb_count = 1;
    }

    // camera init
    esp_err_t err = esp_camera_init(&config);
    if (err != ESP_OK) {
        Serial.printf("Camera init failed with error 0x%x", err);
        delay(1000);
        ESP.restart();
    }
}

bool parseSensorData(String data) {
    int parsed = sscanf(data.c_str(), "Temperature: %f°C Humidity:
%f%% PPM : %f ppm",
                        &temperature, &humidity, &correctedPPM);
    return parsed == 3; // Return true jika parsing berhasil
}

void handleNewMessages(int numNewMessages) {

```

```

Serial.print("Handle New Messages: ");
Serial.println(numNewMessages);

for (int i = 0; i < numNewMessages; i++) {
    String chat_id = String(bot.messages[i].chat_id);
    if (chat_id != CHAT_ID) {
        bot.sendMessage(chat_id, "Unauthorized user", "");
        continue;
    }

    // Print the received message
    String text = bot.messages[i].text;
    Serial.println(text);

    String from_name = bot.messages[i].from_name;
    if (text == "/start") {
        String welcome = "Welcome , " + from_name + "\n";
        welcome += "Use the following commands to interact with the
ESP32-CAM \n";
        welcome += "/photo : takes a new photo\n";
        welcome += "/flash : toggles flash LED \n";
        welcome += "/conditions : get temperature and humidity
readings from DHT11 sensor and PPM reading from MQ135 sensor\n";
        bot.sendMessage(CHAT_ID, welcome, "");
    }
    if (text == "/flash") {
        flashState = !flashState;
        digitalWrite(FLASH_LED_PIN, flashState);
        Serial.println("Change flash LED state");
    }
    if (text == "/photo") {
        sendPhoto = true;
        Serial.println("New photo request");
    }
    if (text == "/conditions") {
        String sensorData = "Temperature: " + String(temperature) +
        "°C\n"
                           "Humidity: " + String(humidity) + "%\n"
                           "PPM: " + String(correctedPPM) + " ppm";
        bot.sendMessage(CHAT_ID, sensorData, "");
    }
}
}

String sendPhotoTelegram() {
    const char* myDomain = "api.telegram.org";
    String getAll = "";
    String getBody = "";

    //Dispose first picture because of bad quality
    camera_fb_t * fb = NULL;
    fb = esp_camera_fb_get();
    esp_camera_fb_return(fb); // dispose the buffered image

    //Take a new photo
    fb = NULL;

```

```

fb = esp_camera_fb_get();
if (!fb) {
    Serial.println("Camera capture failed");
    delay(1000);
    ESP.restart();
    return "Camera capture failed";
}

Serial.println("Connect to " + String(myDomain));

if (clientTCP.connect(myDomain, 443)) {
    Serial.println("Connection successful");

    String head = "--RandomNerdTutorials\r\nContent-Disposition:
form-data; name=\"chat_id\"; \r\n\r\n" + CHAT_ID + "\r\n--
RandomNerdTutorials\r\nContent-Disposition: form-data;
name=\"photo\"; filename=\"esp32-cam.jpg\"\r\nContent-Type:
image/jpeg\r\n\r\n";
    String tail = "\r\n--RandomNerdTutorials--\r\n";

    size_t imageLen = fb->len;
    size_t extraLen = head.length() + tail.length();
    size_t totalLen = imageLen + extraLen;

    clientTCP.println("POST /bot" + BOTtoken + "/sendPhoto
HTTP/1.1");
    clientTCP.println("Host: " + String(myDomain));
    clientTCP.println("Content-Length: " + String(totalLen));
    clientTCP.println("Content-Type: multipart/form-data;
boundary=RandomNerdTutorials");
    clientTCP.println();
    clientTCP.print(head);

    uint8_t *fbBuf = fb->buf;
    size_t fbLen = fb->len;
    for (size_t n = 0; n < fbLen; n = n + 1024) {
        if (n + 1024 < fbLen) {
            clientTCP.write(fbBuf, 1024);
            fbBuf += 1024;
        }
        else if (fbLen % 1024 > 0) {
            size_t remainder = fbLen % 1024;
            clientTCP.write(fbBuf, remainder);
        }
    }

    clientTCP.print(tail);

    esp_camera_fb_return(fb);

    int waitTime = 10000;    // timeout 10 seconds
    long startTimer = millis();
    boolean state = false;

    while ((startTimer + waitTime) > millis()) {
        Serial.print(".");
    }
}

```

```

        delay(100);
        while (clientTCP.available()) {
            char c = clientTCP.read();
            if (state == true) getBody += String(c);
            if (c == '\n') {
                if (getAll.length() == 0) state = true;
                getAll = "";
            } else if (c != '\r')
                getAll += String(c);
            startTimer = millis();
        }
        if (getBody.length() > 0) break;
    }
    clientTCP.stop();
    Serial.println(getBody);
}
else {
    getBody = "Connected to api.telegram.org failed.";
    Serial.println("Connected to api.telegram.org failed.");
}
return getBody;
}

void setup() {
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);
    // Init Serial Monitor
    Serial.begin(115200);

    // Init Serial2 for communication with ESP32
    MySerial.begin(115200, SERIAL_8N1, 3, 1); // RX=16, TX=17

    pinMode(FLASH_LED_PIN, OUTPUT);
    digitalWrite(FLASH_LED_PIN, flashState);

    configInitCamera();

    WiFi.mode(WIFI_STA);
    Serial.println();
    Serial.print("Connecting to ");
    Serial.println(ssid);
    WiFi.begin(ssid, password);
    clientTCP.setCACert(TELEGRAM_CERTIFICATE_ROOT);
    while (WiFi.status() != WL_CONNECTED) {
        Serial.print(".");
        delay(500);
    }
    Serial.println();
    Serial.print("ESP32-CAM IP Address: ");
    Serial.println(WiFi.localIP());
}

void loop() {
    if (sendPhoto) {
        Serial.println("Preparing photo");
        sendPhotoTelegram();
        sendPhoto = false;
    }
}

```

```
}

if (MySerial.available() > 0) {
    serialData = MySerial.readStringUntil('\n');
    if (parseSensorData(serialData)) {
        Serial.println("Sensor data updated");
    } else {
        Serial.println("Error parsing sensor data");
    }
}

if (millis() > lastTimeBotRan + botRequestDelay) {
    int numNewMessages = bot.getUpdates(bot.last_message_received +
1);
    while (numNewMessages) {
        Serial.println("got response");
        handleNewMessages(numNewMessages);
        numNewMessages = bot.getUpdates(bot.last_message_received +
1);
    }
}
}
```

```

camera_code

bool setupCamera()
{
    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;
    config.pin_xclk = XCLK_GPIO_NUM;
    config.pin_pclk = PCLK_GPIO_NUM;
    config.pin_vsync = VSYNC_GPIO_NUM;
    config.pin_href = HREF_GPIO_NUM;
    config.pin_sscb_sda = SIOD_GPIO_NUM;
    config.pin_sscb_scl = SIOC_GPIO_NUM;
    config.pin_pwdn = PWDN_GPIO_NUM;
    config.pin_reset = RESET_GPIO_NUM;
    config.xclk_freq_hz = 20000000;
    config.pixel_format = PIXFORMAT_JPEG;
    //init with high specs to pre-allocate larger buffers
    if (psramFound())
    {
        config.frame_size = FRAMESIZE_UXGA; // FRAMESIZE_ +
QVGA|CIF|VGA|SVGA|XGA|SXGA|UXGA
        config.jpeg_quality = 10;
        config.fb_count = 2;

```

```

    }
else
{
    config.frame_size = FRAMESIZE_SVGA;
    config.jpeg_quality = 12;
    config.fb_count = 1;
}

#ifdef CAMERA_MODEL_ESP_EYE
    pinMode(13, INPUT_PULLUP);
    pinMode(14, INPUT_PULLUP);
#endif

// camera init
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK)
{
    Serial.printf("Camera init failed with error 0x%x", err);
    return false;
}

sensor_t *s = esp_camera_sensor_get();

//initial sensors are flipped vertically and colors are a bit
saturated
if (s->id.PID == OV3660_PID)
{
    s->set_vflip(s, 1);          //flip it back
    s->set_brightness(s, 1);    //up the blightness just a bit
    s->set_saturation(s, -2);    //lower the saturation
}

//drop down frame size for higher initial frame rate

```

```
s->set_framesize(s, FRAME_SIZE_QVGA);

#if defined(CAMERA_MODEL_M5STACK_WIDE)
    s->set_vflip(s, 1);
    s->set_hmirror(s, 1);
#endif

return true;
}
```

camera_pins

```
#if defined(CAMERA_MODEL_WROVER_KIT)
```

```
#define PWDN_GPIO_NUM -1
```

```
#define RESET_GPIO_NUM -1
```

```
#define XCLK_GPIO_NUM 21
```

```
#define SIOD_GPIO_NUM 26
```

```
#define SIOC_GPIO_NUM 27
```

```
#define Y9_GPIO_NUM 35
```

```
#define Y8_GPIO_NUM 34
```

```
#define Y7_GPIO_NUM 39
```

```
#define Y6_GPIO_NUM 36
```

```
#define Y5_GPIO_NUM 19
```

```
#define Y4_GPIO_NUM 18
```

```
#define Y3_GPIO_NUM 5
```

```
#define Y2_GPIO_NUM 4
```

```
#define VSYNC_GPIO_NUM 25
```

```
#define HREF_GPIO_NUM 23
```

```
#define PCLK_GPIO_NUM 22
```

```
#elif defined(CAMERA_MODEL_ESP_EYE)
```

```
#define PWDN_GPIO_NUM -1
```

```
#define RESET_GPIO_NUM -1
```

```
#define XCLK_GPIO_NUM 4
```

```
#define SIOD_GPIO_NUM 18
```

```
#define SIOC_GPIO_NUM 23
```

```
#define Y9_GPIO_NUM 36
```

```
#define Y8_GPIO_NUM 37
```

```
#define Y7_GPIO_NUM 38
```

```
#define Y6_GPIO_NUM 39
```

```
#define Y5_GPIO_NUM 35
#define Y4_GPIO_NUM 14
#define Y3_GPIO_NUM 13
#define Y2_GPIO_NUM 34
#define VSYNC_GPIO_NUM 5
#define HREF_GPIO_NUM 27
#define PCLK_GPIO_NUM 25

#elif defined(CAMERA_MODEL_M5STACK_PSRAM)
#define PWDN_GPIO_NUM -1
#define RESET_GPIO_NUM 15
#define XCLK_GPIO_NUM 27
#define SIOD_GPIO_NUM 25
#define SIOC_GPIO_NUM 23

#define Y9_GPIO_NUM 19
#define Y8_GPIO_NUM 36
#define Y7_GPIO_NUM 18
#define Y6_GPIO_NUM 39
#define Y5_GPIO_NUM 5
#define Y4_GPIO_NUM 34
#define Y3_GPIO_NUM 35
#define Y2_GPIO_NUM 32
#define VSYNC_GPIO_NUM 22
#define HREF_GPIO_NUM 26
#define PCLK_GPIO_NUM 21

#elif defined(CAMERA_MODEL_M5STACK_WIDE)
#define PWDN_GPIO_NUM -1
#define RESET_GPIO_NUM 15
#define XCLK_GPIO_NUM 27
```

```
#define SIOD_GPIO_NUM 22
#define SIOC_GPIO_NUM 23

#define Y9_GPIO_NUM 19
#define Y8_GPIO_NUM 36
#define Y7_GPIO_NUM 18
#define Y6_GPIO_NUM 39
#define Y5_GPIO_NUM 5
#define Y4_GPIO_NUM 34
#define Y3_GPIO_NUM 35
#define Y2_GPIO_NUM 32
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM 26
#define PCLK_GPIO_NUM 21

#elif defined(CAMERA_MODEL_AI_THINKER)
#define PWDN_GPIO_NUM 32
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM 0
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27

#define Y9_GPIO_NUM 35
#define Y8_GPIO_NUM 34
#define Y7_GPIO_NUM 39
#define Y6_GPIO_NUM 36
#define Y5_GPIO_NUM 21
#define Y4_GPIO_NUM 19
#define Y3_GPIO_NUM 18
#define Y2_GPIO_NUM 5
#define VSYNC_GPIO_NUM 25
```

```
#define HREF_GPIO_NUM 23
```

```
#define PCLK_GPIO_NUM 22
```

```
#else
```

```
#error "Camera model not selected"
```

```
#endif
```
